BombLab 实验报告

Phase1

（1）推理：

%rdi 存的是传进来的 challenge。shr $0x20, %rax：右移 32 位，提取出 high（高 32 位），shr $0x10, %rsi：右移 16 位，提取出 mid（中间 32 位），剩下的 %edx：就是没动过的 low（低 32 位）。接下来看调用的三个函数。p1_step1：输入 high, mid，返回(high * 3) ^ (mid + 0x6c4b)。p1_step0：输入 low,high，返回(low - high) * 3 + 0x299c。p1_step2：输入前两步的返回值 ret0,ret1，返回(ret0 * 5 + ret1) ^ 0x3432。

（2）关键反汇编：

```
uint32_t high = (uint32_t)(challenge >> 32);
uint32_t low  = (uint32_t)(challenge & 0xFFFFFFFF);
uint32_t mid  = (uint32_t)(challenge >> 16);
uint32_t ret1 = (high * 3) ^ (mid + 0x6c4b);
uint32_t ret0 = (low - high) * 3 + 0x299c;
uint32_t result = (ret0 * 5 + ret1) ^ 0x3432;
```

最后 values[0] = result;count = 1;

（3）运行效果：

公开 seed 验证：

```
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
```

其他 seed 验证：

```
hzixin@Bu:~/BombLab_hzixin$ make
./defuser --seed 888 --phase 1 | ./bomb --seed 888 --phase 1
make: Nothing to be done for 'all'.
```

```
Phase 1 challenge: 0xb341b62109203dd7
Input (unsigned decimal integers):
Phase 1 defused.

[1 已解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

Phase2

（1）推理：

结合栈偏移 -0x40(%rbp) 到 -0x2c(%rbp)，可知输入被存入 uint32_t values[6] 数组中，所以要先定义这个数组。

接下来是一些判断以及根据判断结果决定是否直接返回 0 终止程序。

```
int ret = valid_u32_fields(input, 6);
if (!ret) return 0;
if (sscanf(input, fmt, &values[0], ..., &values[5]) != 6)return 0;
```

回跳 + 计数器 + 上界 = for 循环。循环中存在指令 lea (%rsi,%rsi,4),%esi（等价于 x * 5），shr %cl,%rax（按 shift_amount 右移），以及 and $0x3f（取低 6 位）。循环变量 i 从 1 到 5，shift_amount 初始为 20，每次增加 3。根据上述写出伪代码：

```
if (values[0] != ((challenge >> 33) & 0xFF) + 0x13) fail;
for (i = 1; i < 6; i++) {
    expected = (((challenge >> shift) & 0x3F) + i) ^ (values[i-1] * 5);
    if (values[i] != expected) fail;
    shift += 3;
}
```

（2）关键反汇编：

将伪代码转化成代码，其中判断输入格式部分可以直接舍去，值比较可以直接改写成赋值操作。

```
values[0] = ((challenge >> 33) & 0xFF) + 0x13;
uint32_t shift_amount = 20;
for (size_t i = 1; i < 6; ++i) {
    // 提取 challenge 的指定位，加上索引，再与前一项的 5 倍异或
    uint32_t chunk = ((challenge >> shift_amount) & 0x3F) + i;
    uint32_t prev_val = values[i - 1] * 5;
    values[i] = chunk ^ prev_val;
    shift_amount += 3;
}
```

（3）运行效果：

公开 seed 验证：

```
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
```

其他 seed 验证：

```
hzixin@Bu:~/BombLab_hzixin$ make
./defuser --seed 888 --phase 2 | ./bomb --seed 888 --phase 2
make: Nothing to be done for 'all'.
```

```
Phase 2 challenge: 0x607337e7d7929ac6
Input (unsigned decimal integers):
Phase 2 defused.

[1 待解除] [2 已解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

Phase3

（1）推理：

首先非常容易可得 p3_case0 到 p3_case7 的返回值。接下来对 p3_dispatch 进行处理。先得 if (idx > 7u)return 0;然后输入指令 gdb -q ./bomb 后输入(gdb) x/8a 0x405820 读取跳转表，按照从左到右从上到下的顺序依次与 case0 到 case7 对应。

0x405820: 0x402278 <p3_dispatch+40> 0x402280 <p3_dispatch+48>
0x405830: 0x402288 <p3_dispatch+56> 0x402290 <p3_dispatch+64>
0x405840: 0x402298 <p3_dispatch+72> 0x4022a0 <p3_dispatch+80>
0x405850: 0x402268 <p3_dispatch+24> 0x402270 <p3_dispatch+32>

结果整理为：

```
case 0: return p3_case_0(x);
case 1: return p3_case_3(x);
case 2: return p3_case_4(x);
case 3: return p3_case_1(x);
case 4: return p3_case_2(x);
case 5: return p3_case_7(x);
case 6: return p3_case_6(x);
case 7: return p3_case_5(x);
```

最后处理 phase_03。

```
uint32_t v[2];
if (!valid_u32_fields(input, 2))
return 0;
if (sscanf(input, fmt, &v[0], &v[1]) != 2)
return 0;
uint32_t idx = (uint32_t)challenge & 7u;
if (v[0] != idx)return 0;
uint32_t x =(uint32_t)(challenge >> 32) ^(uint32_t)(challenge >> 3);
uint32_t expected = p3_dispatch(idx, x);
```

（2）关键反汇编：将伪代码转化成代码，其中判断输入格式部分可以直接舍去，值比较可以直接改写成赋值操作。

（3）运行效果：

公开 seed 验证：

```
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
```

其他 seed 验证：

```
hzixin@Bu:~/BombLab_hzixin$ make
./defuser --seed 888 --phase 3 | ./bomb --seed 888 --phase 3
make: Nothing to be done for 'all'.
```

```
Phase 3 challenge: 0x9da7b27d1fd6b456
Input (unsigned decimal integers):
Phase 3 defused.

[1 待解除] [2 待解除] [3 已解除] [4 待解除] [5 待解除] [6 待解除]
```

Phase4

（1）推理：

首先处理 phase_04 的主函数部分。从汇编中可知，程序会校验并解析 1 个整数存入 -0x14(%rbp)（即 values[0]）。接着，程序从 challenge 中提取出三个关键参数用于调用核心函数 p4_walk：target = (uint32_t)challenge & 0x7f；key = (challenge >> 7) ^ (challenge >> 32)

初始搜索范围 lo = 0，hi = 0x7f。然后调用 p4_walk(target, lo, hi, key)，并与用户输入 values[0] 进行比对。

关于输入格式校验的省略：

在书写求解器代码时，类似于前三题，valid_u32_fields 以及 sscanf 检查输入格式的汇编部分可以直接省略不看，因为我们不再需要解析用户输入，而是直接构造输出。

关于 p4_walk 的递归分析与“自创数组压栈”：

p4_walk 本质上是一个带有状态保存的二分查找递归函数。它的逻辑是：计算 mid = lo + (hi - lo) / 2。若 target == mid（触底），直接返回 (mid + 0x83) ^ key。若 target > mid，递归调用右分支，返回后执行 ((ret_val * 5) ^ (mid + key)) ^ 0xe。若 target < mid，递归调用左分支，返回后执行 ((ret_val * 5) ^ (mid + key)) ^ 0xb。

但在 C 语言求解器只能写在单函数内部的限制下，所以选择自己定义 mids[32] 和 dirs[32] 两个数组记录向下的路径，再用 while 循环逆序回溯，从而用迭代替代了递归。

（2）关键反汇编：

将伪代码转化成代码，其中判断输入格式部分可以直接舍去，值比较可以直接改写成赋值操作。

```
// ...（省略了 challenge 的参数提取部分）
uint32_t lo = 0, hi = 0x7f;
uint32_t mids[32];
unsigned char dirs[32];   /* 自建软件栈：1 走右，0 走左 */
size_t depth = 0;

// 阶段一：向下钻（模拟递归压栈）
for (;;) {
    uint32_t mid = lo + ((hi - lo) >> 1);
    if (target == mid) break; // 触底

    mids[depth] = mid;        // 记录当前层的 mid
    if (target > mid) {
        dirs[depth] = 1;
        lo = mid + 1;
    } else {
        dirs[depth] = 0;
        hi = mid - 1;
    }
    ++depth;
}
```

```
    // 阶段二：触底返回 Base Case
    uint32_t val = (mid + 0x83u) ^ key;

    // 阶段三：向上回溯（模拟递归出栈，逆序计算）
    while (depth > 0) {
        --depth;
        uint32_t ebx = mids[depth] + key;
        val = (val * 5u) ^ ebx;
        if (dirs[depth]) {
            val ^= 0xeu;   /* 右分支异或 0xe */
        } else {
            val ^= 0xbu;   /* 左分支异或 0xb */
        }
    }
    values[0] = val;
    count = 1;
```

（3）运行效果：

公开 seed 验证：

```
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
```

其他 seed 验证：

```
hzixin@Bu:~/BombLab_hzixin$ make
./defuser --seed 888 --phase 4 | ./bomb --seed 888 --phase 4
make: Nothing to be done for 'all'.
```

```
Phase 4 challenge: 0xc74a271493865a76
Input (unsigned decimal integers):
Phase 4 defused.

[1 待解除] [2 待解除] [3 待解除] [4 已解除] [5 待解除] [6 待解除]
```

Phase5

（1）推理：

1. 主函数的局部变量与校验省略：

在 phase_05 主函数中，通过 sscanf 的传参规则可知，-0x1c(%rbp) 和 -0x18(%rbp) 分别接收输入的第 1 和第 2 个整数，它们构成局部数组 values[2]。接着，程序执行 lea -0x14(%rbp), %rsi，将一个栈上的局部变量的地址作为输出指针传给遍历函数。此时，p5_traverse 会将算出的部分结果写入该地址，带回主函数。

注意：与前三题一样，这里的 valid_u32_fields 和 sscanf 等防御性格式校验代码，在编写求解器时可以直接省略，因为求解器不需要解析用户输入，而是直接构造答案。

2. 识别全局数组与哈希种子：

在 p5_traverse 开头，指令 mov 0x405b60(,%rax,8), %rdx 表明程序从地址 0x405b60 处按索引取出了一个 8 字节的指针。这个远超栈范围的绝对地址，说明它是一个全局数组的基址（即 heads 数组）。随后，程序将 challenge 移位并异或 0xcd71，算出了一个初始状态值，这个值就是整个运算的“哈希种子”，后续的所有操作都在此基础上滚雪球。

3. 识别结构体与循环：

通过解析偏移量 (%rdx)、4(%rdx) 和 8(%rdx)，可以断定 %rdx 指向一个结构体：偏移量是等差的，但是所存的数据字节长度却不一样（前两个是 32 位，第三个是 64 位，这说明这是一个结构体而不是一个数组）。所以偏移 8 处存放的是一个 64 位的指针（地址），而不是普通的 32 位整数。

在循环部分，je 40247f 是一个前置判断：如果步数为 0 直接退出。而 jae 402467 结合前面的 cmp %ecx, %edi 和计数器的自增，说明这是一个典型的 for 循环，循环次数由 (challenge >> 3) & 3 决定。每次循环顺着 next 指针往后走，并更新哈希状态。

4. 寄存器到变量的映射：

在逆向过程中，我们将汇编中的寄存器映射为 C 语言的变量：

%rdx：一直随着 +8 偏移移动，对应当前的链表节点指针 cur。

%eax：不断参与乘法和异或，对应滚动的累计状态 val。

%r8：一直用于写内存，对应输出指针 out_val。

%ecx：递增并用于比较，对应循环计数器 steps。

5. 数据提取与伪代码转代码：

因为数据存在静态内存中，必须借助 GDB 提取。使用命令 x/8gx 0x405b60 读取头指针数组，再顺藤摸瓜（如 x/4gx 0x...）找出全部 12 个节点的数据，硬编码为 C 代码中的 static const struct nodes[12] 和 heads[8] 数组。

最终，将原汇编中“计算然后比较”的逻辑，反转改写为“直接计算并赋值”的构造逻辑。

（2）关键反汇编：

```
    uint32_t idx = (uint32_t)challenge & 0x7u;
    int cur = heads[idx];
    uint32_t eax = ((uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32)) ^ 0xcd71u;

    // 初始状态计算
    uint32_t f0 = nodes[cur].field0;
    uint32_t f4 = nodes[cur].field4;
    eax = eax * 3u + f0;
    eax ^= f4;
    uint32_t outval = f4;

    // 提取循环步数，使用 for 循环重构原汇编的遍历过程
    uint32_t steps = ((uint32_t)(challenge >> 3)) & 0x3u;
    for (uint32_t i = 0; i < steps; ++i) {
        cur = nodes[cur].next;
        f0 = nodes[cur].field0;
        f4 = nodes[cur].field4;
        eax = eax * 3u + f0;
        eax ^= f4;
        outval = f4;
    }

    values[0] = eax;     // 对应原程序的返回值
    values[1] = outval;  // 对应原程序通过指针写回的值
    count = 2;
```

（3）运行效果：

公开 seed 验证

```
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
```

其他 seed 验证

```
hzixin@Bu:~/BombLab_hzixin$ make
./defuser --seed 888 --phase 5 | ./bomb --seed 888 --phase 5
make: Nothing to be done for 'all'.
```

```
Phase 5 challenge: 0x27984ce988dcb44e
Input (unsigned decimal integers):
Phase 5 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 已解除] [6 待解除]
```

Phase6

(1)推理

1. 省略输入校验：

与之前一致，valid_u32_fields 和 sscanf 等解析用户输入的汇编指令在求解器中直接省略，因为我们只需构造答案。

2. 计算基础种子值：

在 p6_dispatch 中，程序首先将 challenge 切碎并异或，得到一个初始状态：x = (challenge >> 6) ^ (challenge >> 32)。

3. 第一轮函数调度：4025a3: and $0x7, %eax

4025a6: call *0x405e00(,%rax,8)这里 0x405e00 是一个全局的函数指针数组。程序提取 challenge & 7 作为索引查表，调用对应的 p6_op_* 函数对 x 进行处理。返回后，执行 y = 返回值 ^ 0x88a1 作为第二轮调用的输入。

4. 第二轮函数调度：4025a0: and $0x38, %ebx

4025b4: call *0x405e00(%rbx)这是本关最巧妙的地方：0x38 换算成二进制是 0011 1000。challenge & 0x38 其实等价于取 challenge 的第 3、4、5 位，再乘以 8（因为指针占 8 字节）。这相当于用 (challenge >> 3) & 7 作为新索引，再次查询同一个函数表，对第一轮的结果进行二次运算。最后将结果加上偏移量 0x94c。

5. 底层位运算函数的内联与转化：

汇编中的各个 p6_op_* 函数使用了底层的循环移位（rol/ror）和算术指令。在 C 代码中，并没有直接对应循环移位的运算符,必须通过移位拼接来完美复刻。例如 p6_op_2 中的 rol $0x7 被还原为 ((x << 7) | (x >> (32 - 7)))。同时，将查表调用的过程直接“内联”为了两个 switch-case 结构。

（2）关键反汇编：

```c
int solve_phase_6(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 0;
    (void)challenge;
    // TODO: 根据 phase_06 的汇编，用 challenge 算出口令中的各个整数
    uint32_t x =
        (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);
    uint32_t y = 0;
    uint32_t op = (uint32_t)challenge & 0x7u;
    switch (op) {
    case 0:
    case 1:
        /* p6_op_2: rol $0x7; xor $0x6c87 */
        y = ((x << 7) | (x >> (32 - 7))) ^ 0x6c87u;
        break;
    case 2:
        /* p6_op_5: (x ^ 0x18d1) + (x << 5) */
        y = (x ^ 0x18d1u) + (x << 5);
        break;
    case 3:
        /* p6_op_3: ((x << 5) + 0xc20c) ^ (7*x) */
        y = ((x << 5) + 0xc20cu) ^ (x * 7u);
        break;
    case 4:
        /* p6_op_4: (x ^ (x >> 5)) + 0xc22e */
        y = (x ^ (x >> 5)) + 0xc22eu;
        break;
    case 5:
        /* p6_op_0: ror $0x5 */
        y = (x >> 5) | (x << (32 - 5));
        break;
    case 6:
    case 7:
        /* p6_op_1: (x + 0x9d0) ^ (x >> 2) */
        y = (x + 0x9d0u) ^ (x >> 2);
```

```
        y = (x + 0x9d0u) ^ (x >> 2);
        break;
    default:
        y = x;
        break;
    }
    y ^= 0x88a1u;
    x = y;
    op = ((uint32_t)challenge >> 3) & 0x7u;
    switch (op) {
    case 0:
    case 1:
        y = ((x << 7) | (x >> (32 - 7))) ^ 0x6c87u;
        break;
    case 2:
        y = (x ^ 0x18d1u) + (x << 5);
        break;
    case 3:
        y = ((x << 5) + 0xc20cu) ^ (x * 7u);
        break;
    case 4:
        y = (x ^ (x >> 5)) + 0xc22eu;
        break;
    case 5:
        y = (x >> 5) | (x << (32 - 5));
        break;
    case 6:
    case 7:
        y = (x + 0x9d0u) ^ (x >> 2);
        break;
    default:
        y = x;
        break;
    }
```

（3）运行效果：

公开 seed 验证：

```
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
```

其他 seed 验证：

```
hzixin@Bu:~/BombLab_hzixin$ make
./defuser --seed 888 --phase 6 | ./bomb --seed 888 --phase 6
make: Nothing to be done for 'all'.
```

```
Phase 6 challenge: 0xc06108c4a854fcb9
Input (unsigned decimal integers):
Phase 6 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 已解除]
```

从伪代码到代码

1. 核心逻辑的反转：从“校验”到“构造”

验证器的逻辑（伪代码）：它面对的是一个未知数（用户输入 V）。它的任务是判断 V 是否等于某个期望值 E。在汇编中表现为：计算 E→比较 V 和 E → 相等则放行，不等则爆炸。

生成器的逻辑（代码）：求解器面对的是已知条件（challenge），目标是生成正确的 V。既然 E 是完全由 challenge 决定的确定值，那我们直接计算出 E，并令 V=E，就必然能通过验证。我们跳过了“猜测”和“验证”的环节，直接走向了“真理”。

伪代码（验证视角）：`if (input_val != expected_val) { return 0; }`

变成代码（构造视角）：`input_val = expected_val;` （直接赋值）

案例：Phase 2 中，伪代码是 `if (values[i] != (...))`，实际代码直接改为 `values[i] = ...`，把验证公式变成了赋值公式。

2. 控制流的重组与内联

汇编代码为了实现模块化和复用，经常把功能拆分成子函数，或者使用跳转表（Jump Table）。在写高级语言求解器时，为了代码紧凑，常常会进行“内联”和“扁平化”。

伪代码：`switch(idx) { case 0: return p3_case_0(x); }` （保留了函数调用结构）

变成代码：直接把 `p3_case_0(x)` 的公式塞进 `switch` 里，如 `ans = (x * 4u) ^ 0x7b41u;`。

案例：Phase 3 求解代码中，直接把 `p3_dispatch` 和底层的 `p3_case_*` 全部合并内联进了

一个大的 switch 语句中。

3. 冗余防御代码的剪枝

逆向出的伪代码往往包含原程序为了安全写的防御性检查,但求解器在特定上下文中可以证明这些检查是多余的。

伪代码：idx = challenge & 7; if (idx > 7) return 0; （保留了原汇编的防御逻辑）

变成代码：直接省略 if (idx > 7)，只保留 switch 的 default: ans = 0;。因为 & 7 从数学上锁死了范围，防御检查永远为假。

4. 寄存器倒腾的合并

汇编指令是一条条执行的（如先算乘法，再存入临时寄存器，再异或）。高级语言可以写成一行复合表达式。

伪代码：temp1 = x * 5;

temp2 = challenge >> shift;

temp3 = temp2 & 0x3F;

temp4 = temp3 + i;

values[i] = temp4 ^ temp1;

变成代码：提取出中间变量增加可读性，或者直接合并成：values[i] = (((challenge >> shift) & 0x3F) + i) ^ (values[i-1] * 5);

5. 控制流模式的规范化

汇编中的回跳（Back-jump）指令被还原成高级语言的 for 或 while。

伪代码（直接描述汇编）：shift = 20; i = 1; loop: ...; i++; shift += 3; if (i != 6) goto loop;

变成代码：转换为清晰的 for 循环 for (size_t i = 1; i < 6; ++i) { ... shift += 3; }